IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Redes neuronales: de la neurona al control

Una red con NumPy, un MLP con Keras y cómo evitar que memorice · un recorrido por S10_01 a S10_03

Sprint 10 · recorrido global de los cuadernos S10_01 a S10_03

DÓNDE ESTAMOS

De los modelos clásicos a las redes neuronales

Hasta ahora (S07 a S09)

Árboles, k-NN, SVM, regresiones, clustering…

Tú decides qué columnas usa cada modelo.

Pocas decisiones de diseño.

Redes neuronales (S10)

Muchas **neuronas** simples conectadas en capas.

La red **aprende sola** qué combinaciones de columnas sirven.

Mucho más flexible… y **más fácil de estropear**.

Lo que cambia

Hay **más ajustes**: capas, neuronas, tasa, épocas.

Hay **más riesgo** de memorizar.

Hay que **comparar siempre** con un modelo sencillo.

Una red neuronal no es magia: es una cuenta de multiplicar, sumar y comprimir, repetida muchas veces.

En S10 usamos los datos de TurisData: dos anillos de juguete y 6 000 reservas (cancelaciones y precio por noche).

LOS TEMAS DE HOY

Tres cuadernos, tres preguntas

Tema	La pregunta que responde	Cuaderno
Red desde cero	¿Qué hay **dentro** de una red neuronal?	S10_01
MLP en Keras	¿Cómo la **construyo y entreno** con datos reales?	S10_02
Regularización	¿Cómo evito que **memorice**?	S10_03

El hilo común: **no te fíes de una cifra sola**. Una red buena es la que **funciona con datos que no ha visto**.

Recorrido: 1 · Red desde cero → 2 · MLP en Keras → 3 · Regularización.

Una neurona: sumar con pesos y comprimir

Una reserva entra en una neurona

Antelación 2,0 × peso 0,5	1,0
Desayuno 1,0 × peso −0,6	-0,6
Sesgo (empujón inicial)	+ 0,2
z = suma ponderada + sesgo	0,6
sigmoide(0,6) = probabilidad de cancelar	**0,646 ≈ 65 %**

Tres pasos

Multiplicar cada entrada por su peso.

Sumar todo y añadir el sesgo.

Comprimir con una activación.

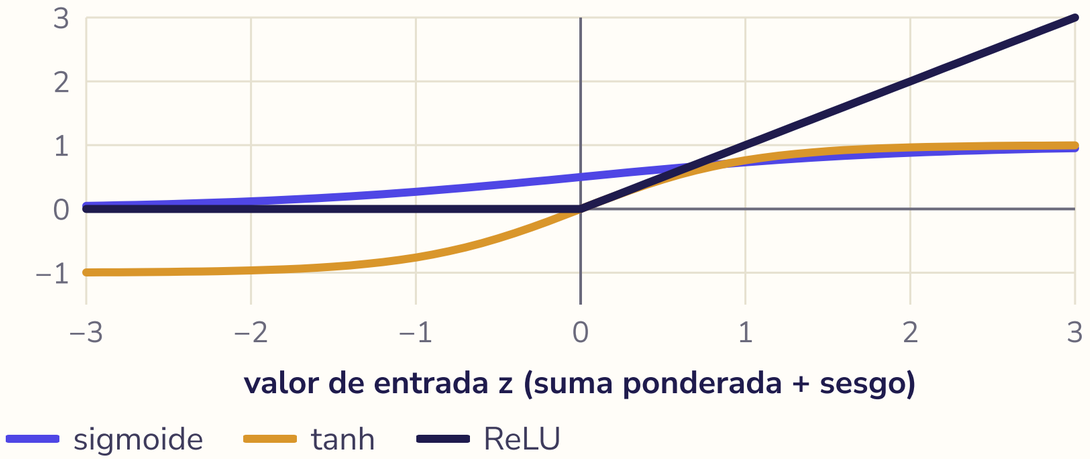
Qué se aprende

Los **pesos** y el **sesgo**. Entrenar es buscar los que dan mejores predicciones.

Una neurona es una **regresión logística pequeña**. Una red es **muchas neuronas** conectadas.

Sin activaciones, apilar capas no sirve

Salida de la activación según z



z	sigmoide	tanh	ReLU
−1,5	0,18	−0,91	0
0	0,50	0,00	0
2,5	0,92	0,99	2,5

Cuál se usa

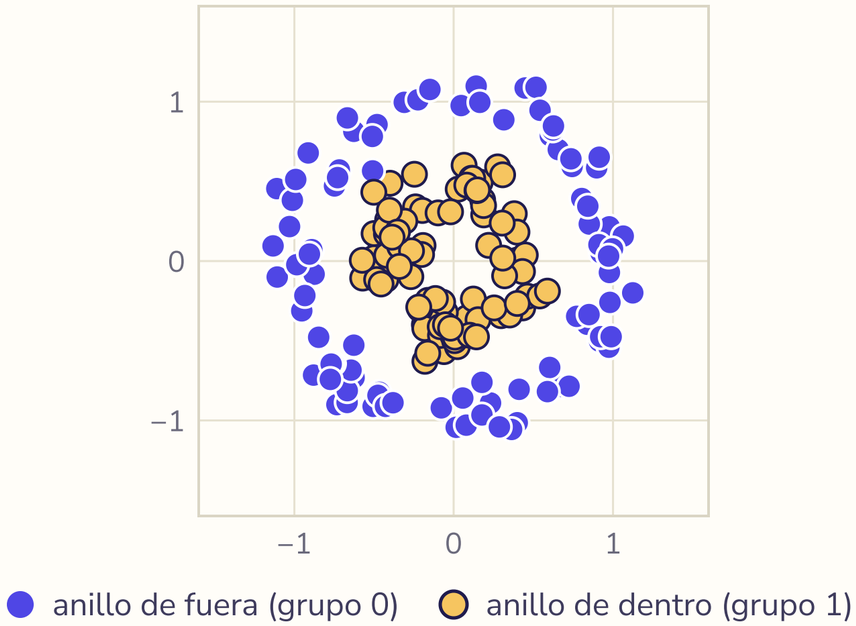
ReLU en las capas ocultas.

Sigmoide en la salida de un sí o no.

Una suma de sumas ponderadas **sigue siendo una suma ponderada**: sin activación, la red es un modelo **lineal**.

Dos anillos que ninguna recta separa

Puntos de dos grupos, uno dentro del otro



Una recta no basta

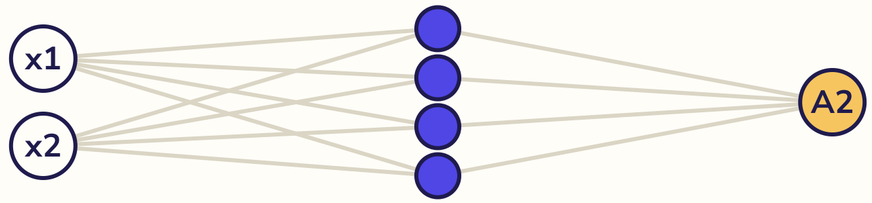
Cualquier recta deja parte de cada anillo a cada lado: un modelo lineal acierta **como el azar** (**0,494**).

La red que construimos

2 entradas → capa oculta (ReLU) → **1 salida** (sigmoide).

Piensa en los anillos como dos tipos de clientes cuyo comportamiento **no se separa con una recta**.

Una capa son muchas neuronas a la vez



Qué entra y qué sale

Entran **2 datos** del punto.

La capa oculta los mezcla con **4 neuronas**.

La salida da una **probabilidad**.

Z1 = X · W1 + b1
suma ponderada de la capa oculta

→

A1 = ReLU(Z1)
activación

→

Z2 = A1 · W2 + b2
suma ponderada de la salida

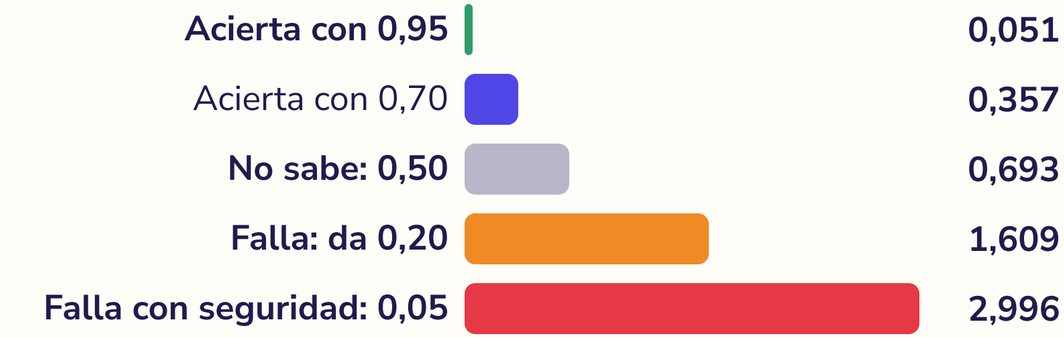
→

A2 = sigmoide(Z2)
probabilidad del grupo 1

Cuatro pasos, de izquierda a derecha: es la **propagación hacia delante**. Una multiplicación de matrices calcula todas las neuronas a la vez.

La pérdida castiga fallar con seguridad

Pérdida según la probabilidad que da a la respuesta correcta



Menos es mejor. Entropía cruzada.

Cómo se lee

Mira la probabilidad que la red dio a **lo correcto**.

0,5 (no saber nada) vale ln 2 = **0,693**.

La clave

Fallar con seguridad se castiga mucho más que acertar con seguridad se premia.

El gradiente: ¿sube o baja el error?

Hacia delante (x = 2, sí cancela)

z1 = 2 · 0,5 + 0	1,00
a1 = ReLU(z1)	1,00
z2 = 1,00 · 1,0 – 0,5	0,50
a2 = sigmoide(z2)	0,622
error = –ln(0,622)	**0,474**

Hacia atrás: gradientes

dz2 = a2 – y	-0,378
dw2 = a1 · dz2	-0,378
dz1 = dz2 · w2	-0,378
dw1 = x · dz1	**-0,755**
db1 = dz1	-0,378

El error de la salida (**a2 – y**) **viaja hacia atrás.** Todos los gradientes son negativos: **subir los pesos baja el error.**

Un pasito en contra del gradiente

Parámetro	Valor	Gradiente	Nuevo = valor − 0,1 · gradiente
w1	0,500	−0,755	0,576
b1	0,000	−0,378	0,038
w2	1,000	−0,378	1,038
b2	−0,500	−0,378	−0,462

0,474 → 0,380

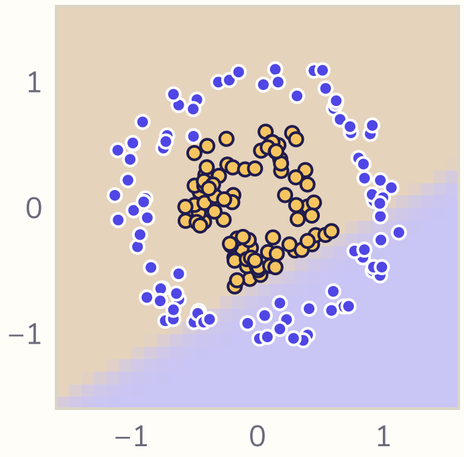
el error de esta reserva baja tras un solo paso

Adelante
calcula la predicción
→
Error
mide cuánto falla
→
Atrás
calcula gradientes
→
Actualizar
valor − tasa · gradiente

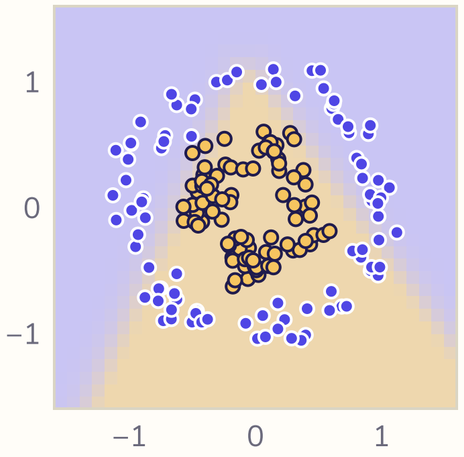
Repetirlo con todos los datos es una **época**. La **tasa de aprendizaje** (aquí 0,1) es el tamaño del pasito.

Más neuronas, una frontera más fina

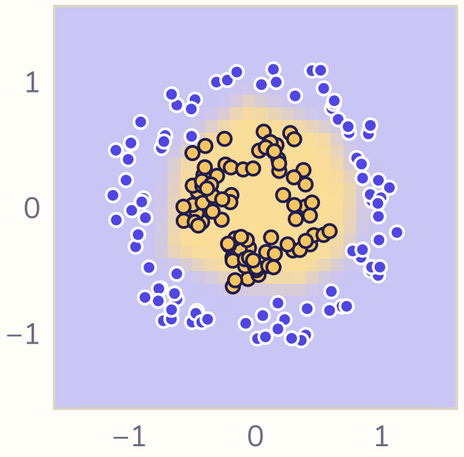
1 neurona oculta · 65,6 %



2 neuronas ocultas · 77,2 %



8 neuronas ocultas · 100 %

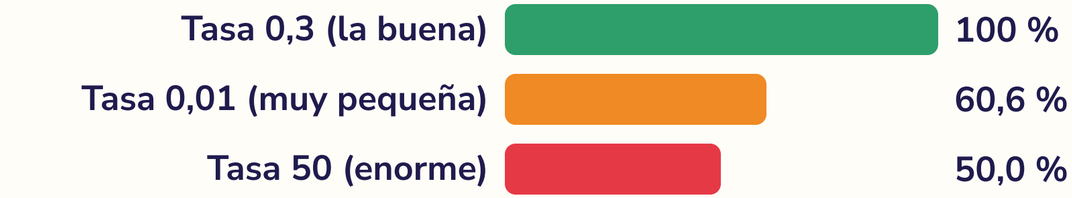


Fondo dorado: la red predice «grupo 1»; fondo lavanda: «grupo 0». Los puntos son de validación.

Con 1 neurona, una recta (65,6 %). Con 8, la red dibuja el anillo y acierta el 100 %. Antes, la recta daba 49,4 %.

La tasa de aprendizaje manda

Exactitud de validación (4 neuronas, 1 500 épocas)



Misma red, mismos datos: solo cambia la tasa.

Muy pequeña: lenta

Con 0,01, la pérdida **apenas ha bajado** tras 1 500 épocas: 0,721 → 0,621.

Enorme: inestable

Con 50, los pesos **saltan** de un lado a otro: la pérdida **sube** (1,122 → 5,944).

Con 0,3, la pérdida baja de 0,716 a 0,025. **No hay una tasa universal**: se prueba y se mira la curva.

Lo que programaste, Keras lo hace por ti

En S10_01, con NumPy	En Keras
Una capa: `X @ W + b` y `relu`	`Dense(32, activation="relu")`
La red completa: `RedNeuronal`	`keras.Sequential([...])`
`entropia_cruzada`	`loss="binary_crossentropy"`
`paso_gradiente`, con tasa 0,3	`optimizer="adam"`
El bucle de épocas: `entrenar`	`modelo.fit(..., epochs=30)`
`atras`: los gradientes, a mano	Automático

Mismas ideas, menos código. Lo difícil ahora son los **datos** y **leer bien las curvas**.

Escalar, codificar y separar sin hacer trampa

Tipo de columna	Ejemplo	Transformación
Numérica	antelación de 150 días (media 100, desv. 50)	**StandardScaler**: (150 – 100) / 50 = **+1,0**
Categórica	canal = agencia (web, agencia, teléfono)	**OneHotEncoder**: web 0 · **agencia 1** · teléfono 0

Las 6 000 reservas, en tres partes



Fuga de datos

Si el escalador **ve** la validación o la prueba, **espía** datos que la red no debe conocer.

Cómo se evita

Se **ajusta solo con entrenamiento** y después se **aplica** al resto.

Un MLP: dos capas ocultas y una salida

Capa	Forma	Parámetros
Oculta 1 (ReLU)	35 → 32	35 · 32 + 32 = **1 152**
Oculta 2 (ReLU)	32 → 16	32 · 16 + 16 = **528**
Salida (sigmoide)	16 → 1	16 + 1 = **17**
Total		**1 697**

Parámetros de una capa

entradas · neuronas + un sesgo por neurona.

Si el total no cuadra, la forma no es la pedida.

La pareja de siempre

Salida **sigmoide** + pérdida **entropía cruzada**: la que programaste a mano.

Sequential apila las capas. **compile** elige **optimizador, pérdida y métricas**. **fit** entrena.

Época, lote y validación

Un ejemplo con números pequeños

Filas de entrenamiento	`1 000`
Tamaño del lote (batch)	`50`
Actualizaciones por época	`1 000 ÷ 50 = 20`
Épocas	`10`
Actualizaciones totales	`10 × 20 = 200`

Época

Una **pasada completa** por las reservas de entrenamiento.

Lote

Las filas que se usan **antes de actualizar** los pesos.

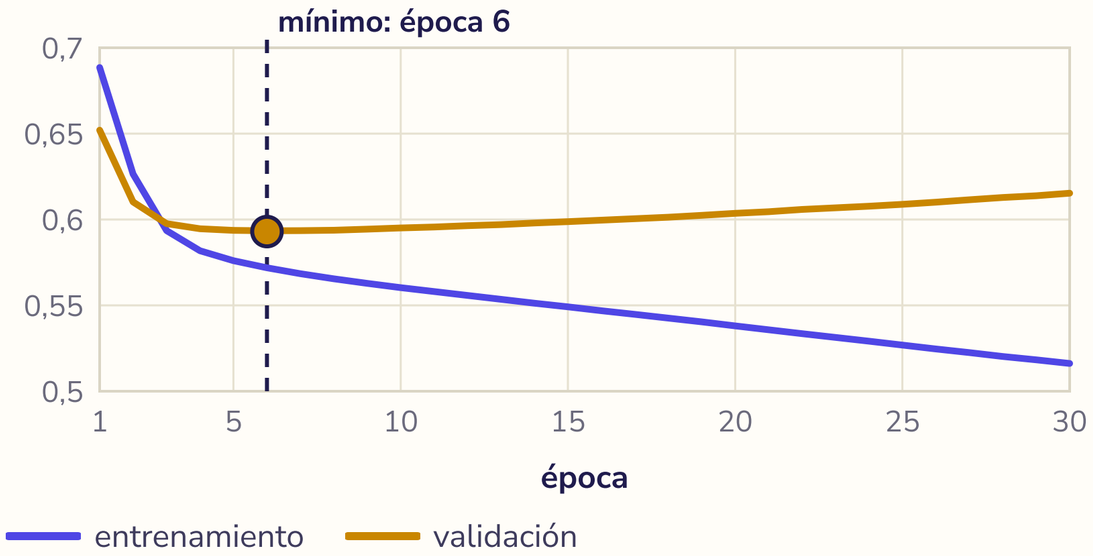
Validación

Se mide en cada época con datos que **no se entrenan**.

Keras guarda la pérdida y las métricas de cada época para **dibujar las curvas**.

Las curvas dicen cuándo empieza a memorizar

Pérdida del MLP (30 épocas, 3 840 reservas)



Entrenamiento

Sigue **bajando siempre**: 0,689 → 0,516.

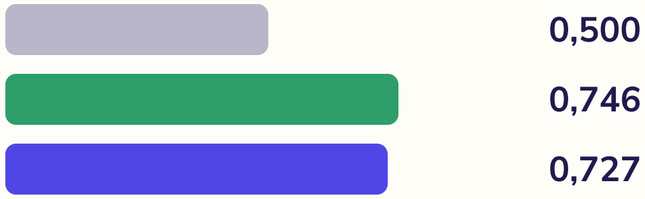
Validación

Toca fondo en la **época 6** (0,593) y **sube** hasta 0,615: **sobreajuste**.

Lo que importa es la curva de **validación**: la de entrenamiento **siempre** parece buena.

¿Gana la red a un modelo sencillo?

AUC en 1 200 reservas de prueba



Más es mejor. 0,5 = no saber nada.

Exactitud

Siempre «no»: **0,541**. Logística: **0,678**. MLP: **0,661**.

Veredicto

mlp_gana es **False**: la red queda **2 centésimas por debajo** de la logística.

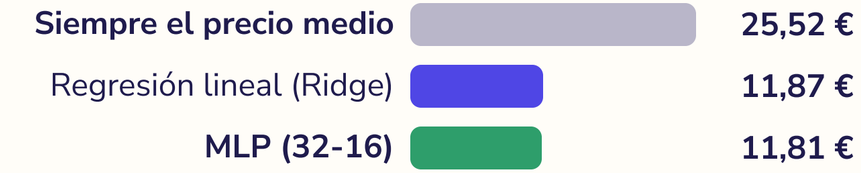
Una red no se justifica por ser una red: tiene que ganar a un modelo sencillo. Aquí no gana.

Acertar un 54 % es lo que consigue quien no sabe nada. Ganar por **menos de 0,02** de AUC es ruido.

Precio por noche: cambia solo la cabeza

	Clasificación	Regresión
Salida	`Dense(1, "sigmoid")`	`Dense(1)`, sin activación
Pérdida	`binary_crossentropy`	`mse` (métrica: `mae`)
Objetivo	0 o 1	precio estandarizado

Error medio absoluto en prueba (€)



Menos es mejor.

El MLP baja el error un **54 %** frente al precio medio… pero **empata con Ridge** (0,06 € de diferencia).

Más parámetros no son un mejor modelo

Tras 15 épocas: la red grande memoriza más y generaliza peor



7,6 veces más parámetros

Más **capacidad**… y más facilidad para **memorizar**.

Cómo decidir

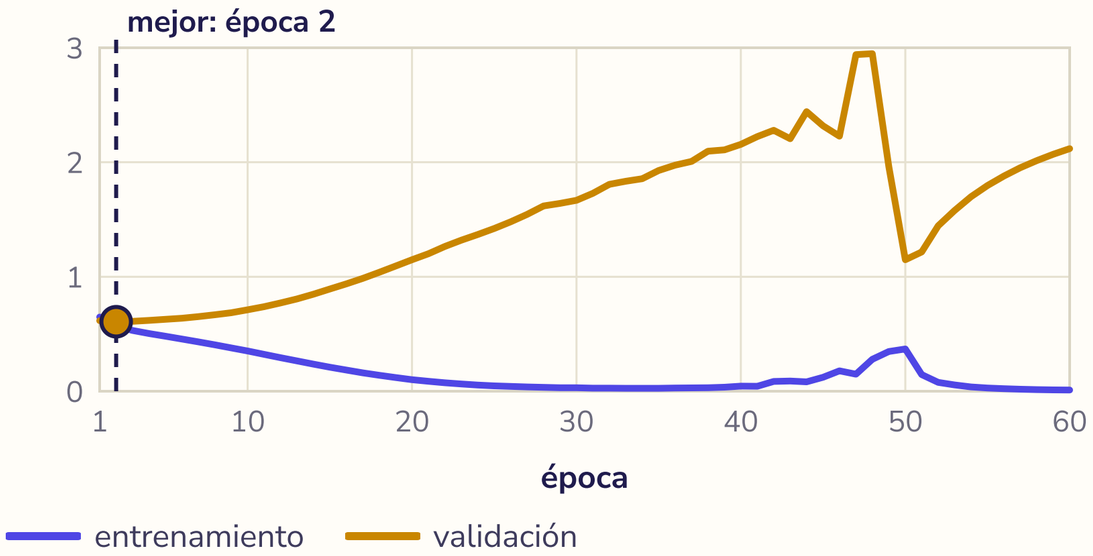
Se mira la curva de **validación**, no la de entrenamiento, y se compara con una **red pequeña**.

Pérdida de validación: **0,599** (red pequeña) frente a **0,637** (red grande). Más parámetros solo ayudan si hay **datos suficientes**.

3 · REGULARIZACIÓN

Una red con muchos parámetros memoriza

Pérdida por época · red 256-128 con solo 800 reservas



42 241

parámetros de la red, para **800** reservas

0,011 vs 2,12

pérdida de entrenamiento al final frente a la de validación

Brecha de generalización = distancia entre las dos curvas. La pérdida de entrenamiento sola **siempre parece buena**.

Cuatro formas de frenar la memorización

Técnica	Idea en una frase	En Keras
L2 (weight decay)	Castiga los **pesos grandes**: la red prefiere soluciones suaves	`kernel_regularizer=l2(0.01)`
Dropout	En cada paso **apaga al azar** un % de neuronas: no depende de una sola	`Dropout(0.3)`
Parada temprana	**Para** cuando la validación deja de mejorar	`EarlyStopping(patience=...)`
Red más pequeña	Menos parámetros: **menos capacidad** para memorizar	`capas=(64, 32)`

Ninguna **crea información**: solo evitan que la red se **fije en el ruido**.

L2 y dropout, con números pequeños

L2 suma una multa a la pérdida

Pesos 3, −0,5, 2: Σ w²	**13,25**
Multa = 0,01 · 13,25	**0,1325**
Pesos 1, −0,5, 2: Σ w²	**5,25**
Multa = 0,01 · 5,25	**0,0525**

Pérdida total = error + λ · Σ w², con λ = 0,01. El peso grande (3) es el que más multa pone.

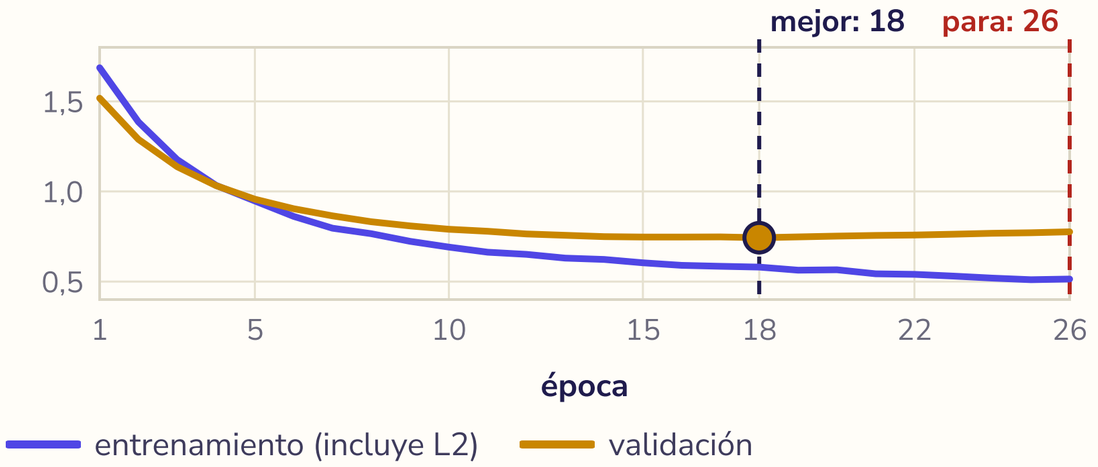
Neurona	n1	n2	n3	n4	n5
activación	0,8	1,5	0,2	1,0	0,6
¿se queda?	sí	no	sí	sí	no
al entrenar	1,33	0	0,33	1,67	0

Dropout 0,4 apaga neuronas al azar. Las que siguen se multiplican por 1 / (1 − 0,4) = **1,67**. Solo actúa **al entrenar**.

L2: la red reduce los pesos grandes. Dropout: la red no depende de una sola neurona.

Parar a tiempo y quedarse con lo mejor

EarlyStopping (paciencia 8) + ReduceLROnPlateau (paciencia 4)



26 / 100

épocas entrenadas, de 100 posibles

restore_best_weights

Devuelve los pesos de la **época 18**, no los de la 26.

Mejor época 18 + paciencia 8 = **26**. La tasa baja de 0,001 a 0,0005 en la época 23 (paciencia 4).

Gradientes que desaparecen o explotan

Capas	×0,25 por capa (se apaga)	×1,5 por capa (se dispara)
2	0,0625	2,25
5	0,00098	7,59
10	0,00000095	57,67

Por qué y síntomas

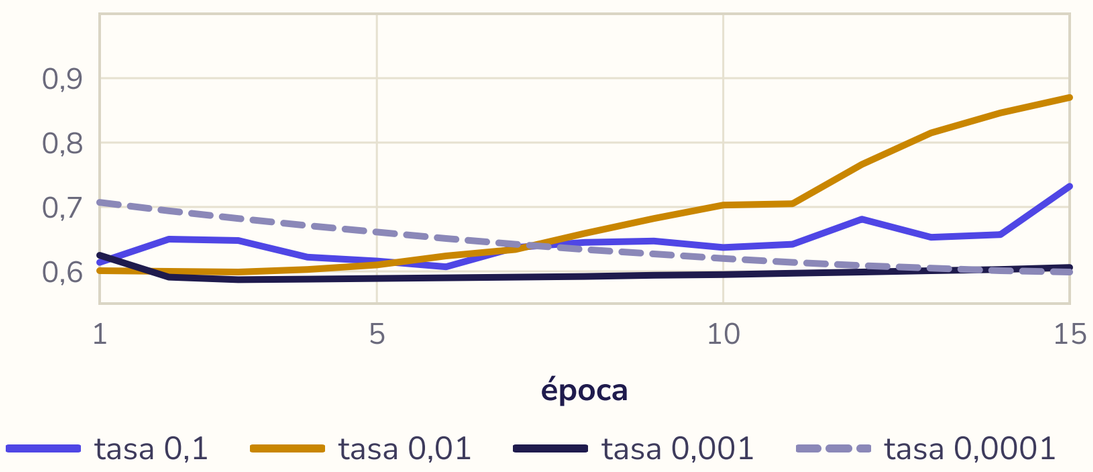
Al retropropagar se **multiplica un factor por capa** (la sigmoide da **0,25 como máximo**).

Se apaga: la pérdida apenas baja. **Se dispara**: salta o sale **nan**.

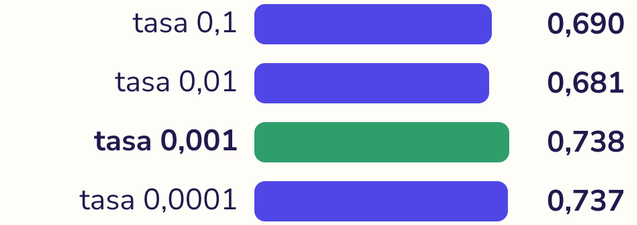
Remedio	En Keras
ReLU (derivada 1 si la entrada es positiva)	`activation="relu"`
Inicialización pensada para ReLU	`kernel_initializer="he_normal"`
Normalizar cada lote	`BatchNormalization()`
Recortar el gradiente	`Adam(clipnorm=1.0)`

Tasa de aprendizaje y tamaño de lote

Pérdida de validación por época



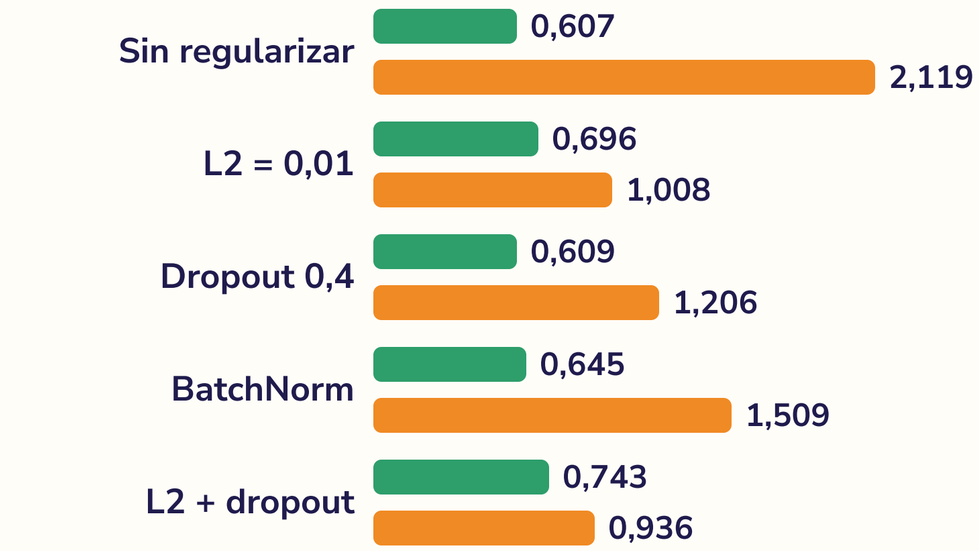
AUC de validación final



3 840 filas, red 64-32, 15 épocas.

Se prueba en **escala logarítmica** (0,1 · 0,01 · 0,001…). Lote pequeño: **más ruido** y más lento. Lote grande: **más rápido**, menos actualizaciones.

Cinco recetas contra el sobreajuste



Bajan la pérdida final

De **2,12** a **0,94–1,51**.

Pero no crean información

El **AUC máximo apenas se mueve** (0,70 a 0,73): son frenos, no milagros.

L2 suma su multa a las pérdidas: por eso su pérdida sale más alta. **Dropout 0,4** deja la validación final en **1,21**.

Guardar el modelo y compararlo con honestidad

Entrenar
con parada temprana

→

Guardar .keras
arquitectura + pesos + optimizador

→

Guardar el preprocesado
escalador y one-hot (joblib)

→

Cargar y comprobar
mismas probabilidades

Mini-reto: pérdida de validación (menos es mejor)



Red grande, sin regularizar (40 épocas) — **1,3336**



(64, 32) + L2 + dropout + parada — **0,6151**

La referencia honesta: la misma red entrenada «a ciegas» hasta el final.

Ojo

Un **.keras** **no guarda** el escalado ni el one-hot: hay que guardarlos aparte.

QUÉ HACER CUANDO LA RED FALLA

Del síntoma al remedio

Lo que ves	Qué pasa	Qué pruebo
Entrenamiento baja, validación **sube**	**Memoriza** (sobreajuste)	L2, dropout, parada temprana, red más pequeña, más datos
La pérdida **apenas baja** o sale `nan`	Gradientes o **tasa**	ReLU, `he_normal`, BatchNorm, `clipnorm`, ajustar la tasa
La curva **salta**	Tasa muy **alta**	Bajar la tasa (escala logarítmica)
La red **no gana** a la logística	Un modelo sencillo basta	**Quedarme con el sencillo**
Quiero **reutilizar** la red	Hay que guardar todo	`.keras` **+ el preprocesado**

Empieza por lo simple; complica solo si la validación lo justifica.

MAPA FINAL

Idea, cómo sabes si funciona y la trampa típica

Idea	Cómo sabes si funciona	La trampa típica
Red desde cero	Los **gradientes** coinciden con el cálculo numérico	Una **tasa** mal elegida (muy pequeña o enorme)
MLP en Keras	**Curvas** de validación y **comparar** con un modelo sencillo	Creer que la red **siempre gana**; **fuga de datos**
Regularización	La **pérdida de validación** deja de subir	Fiarse de la curva de **entrenamiento**

Una red es buena si **funciona con datos que no ha visto** y **gana a un modelo sencillo**.

PARA LLEVARSE

Lo que tienes que poder decir

Red desde cero

Una neurona **suma con pesos y comprime**. Se aprende con **gradientes**.

MLP en Keras

Sequential + **compile** + **fit**. **Escala, codifica y separa** sin fugas.

Regularización

Lee las **curvas de validación**. **L2, dropout y parada temprana**.

Siempre

Compara con un modelo sencillo. Una cifra sola no dice nada.

Los detalles, en las presentaciones S10_01 a S10_03 y en los cuadernos.